drawing a variety of actions from the action probability vector output by our model.

The agents in the genetic algorithm, on the other hand, are not nudged in any direction. We produce a lot of agents in each generation, and with so much random variation between them, most of them will have different policies than each other. There still is an exploration versus exploitation problem in evolutionary strategies because too little mutation can lead to premature convergence where the whole population becomes filled with nearly identical individuals, but it's generally easier to ensure adequate exploration with genetic algorithms than with gradient descent-based ones.

6.4.2. Evolutionary algorithms are incredibly sample intensive

As you could probably see from the code in this chapter, we needed to run each agent in a population of 500 through the environment to determine their fitness. That means we needed to perform 500 major computations before we could make an update to the population. Evolutionary algorithms tend to be more sample hungry than gradient-based methods, since we aren't strategically adjusting the weights of our agents; we are just creating lots of agents and hoping that the random mutations and recombinations we introduce are beneficial. We will say that evolutionary algorithms are less *data-efficient* than DQN or PG methods.

Suppose we want to decrease the size of the population to make the algorithm run faster. If we decrease the population size, there are fewer agents to select from when we are picking the two parents. This will make it likely that less fit individuals will make it into the next generation. We rely on a large number of agents being produced in hopes of finding a combination that leads to better fitness. Additionally, as in biology, mutations usually have a negative impact and lead to worse fitness. Having a larger population increases the probability that at least a few mutations will be beneficial.

Being data-inefficient is a problem if collecting data is expensive, such as in robotics or with autonomous vehicles. Having a robot collect one episode of data usually takes a couple of minutes, and we know from our past algorithms that training a simple agent takes hundreds if not thousands of